Generative AI: From Basics to Intermediate

How to use this report

Generative AI (GenAI) is software that creates new content — text, images, code, audio, video — by learning patterns from huge amounts of existing data. This report takes you from first principles to the techniques practitioners use daily: prompting, RAG, fine-tuning and agents.

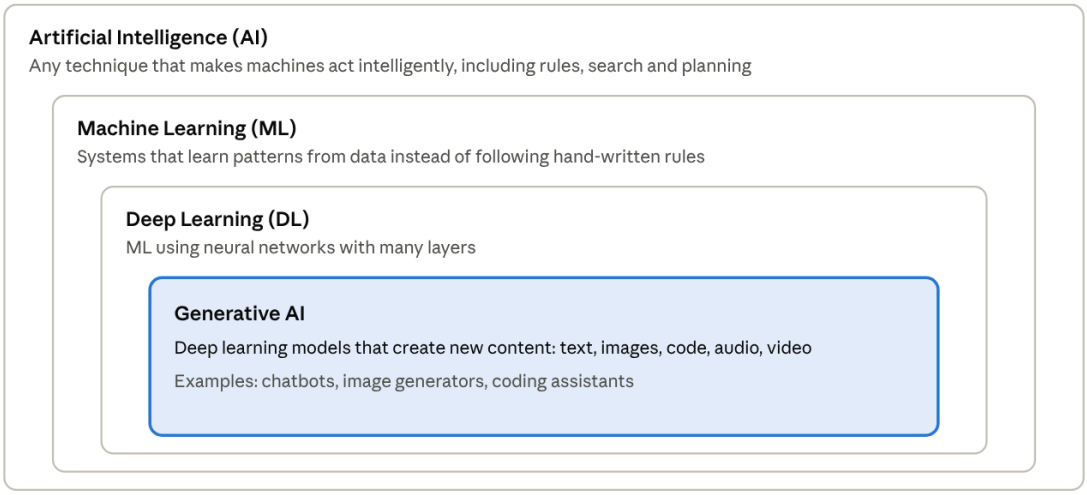
Read it in order; each part builds on the one before. Every part ends with a key idea to remember.

Stage	Parts	You will be able to
Foundations	1–3	Explain what GenAI is, where it came from, and what tokens, embeddings and training mean
How it works	4–6	Compare model families, explain attention, and describe how an LLM is trained
Using it well	7–9	Write effective prompts, design a RAG system, and understand AI agents
Applying it responsibly	10–11	Match modalities to use cases, evaluate outputs, and manage risks
Review	Glossary + quiz	Check your understanding and plan next steps

Part 1 — What is Generative AI?

Generative AI is a type of deep learning model that produces **new** content resembling the data it was trained on. Ask it for a poem, a product photo or a Python function, and it generates one that did not exist before.

Generative AI sits inside deep learning, inside machine learning, inside AI



Where GenAI fits · four nested fields

Each field is a subset of the one around it: all GenAI is deep learning, but not all deep learning is generative.

Generative vs discriminative models

Most classic ML is *discriminative*: it labels or predicts. Generative models instead learn the full pattern of the data well enough to produce new samples.

Aspect	Discriminative AI	Generative AI
Core question	"Which category is this?"	"What would a new example look like?"
What it learns	The boundary between classes, P(label given input)	The data distribution itself, P(data)
Typical output	A label, score or number	Text, image, audio, code, video
Example task	Is this email spam?	Write a reply to this email
Example models	Logistic regression, random forest, image classifiers	GPT-style LLMs, diffusion models, GANs

Key idea: GenAI does not look answers up — it predicts plausible new content from learned patterns. That is both its power and the source of its mistakes.

Part 2 — A short history

GenAI took off after 2017, when the Transformer architecture made it practical to train very large models on internet-scale text. Earlier ideas (GANs, VAEs) proved machines could generate realistic data; Transformers made it general-purpose.

Year	Milestone	Why it mattered
2024–25	Reasoning models, multimodal assistants and AI agents	Models "think" step by step before answering and use tools to complete multi-step tasks
2023	GPT-4, Claude, Llama and other large models	Strong multi-domain ability; open-weight models spread widely
2022	Stable Diffusion, DALL·E 2, ChatGPT	Image and chat generation reach the general public
2020	GPT-3 (175B parameters); DDPM diffusion models	Scale alone unlocked few-shot learning; diffusion became the image standard
2018	GPT-1 and BERT	Pretrain-then-fine-tune becomes the dominant recipe
2017	"Attention Is All You Need" — the Transformer	Parallel training with attention; the basis of every modern LLM
2014	Generative Adversarial Networks (GANs)	First convincingly realistic generated images
2013	Variational Autoencoders (VAEs); word2vec embeddings	Learned latent spaces and word meaning as vectors
1966	ELIZA chatbot	Early rule-based conversation — no learning involved

Key idea: three things drove the boom together — the Transformer architecture, massive datasets, and GPU computing power.

Part 3 — Core building blocks

Every modern GenAI model is a large neural network that turns input into numbers, transforms those numbers, and turns them back into output. Five concepts cover most of what you need.

Neural networks and parameters

A neural network is layers of simple math units ("neurons") connected by adjustable weights called **parameters**. Training nudges billions of these weights until the network's outputs match the training data. More parameters generally means more capacity to learn patterns — and more compute to run.

Tokens

Models don't read words; they read **tokens** — chunks of text, often a word or part of a word. As a rule of thumb, 1 token ≈ ¾ of an English word, so 100 tokens ≈ 75 words. Pricing, speed and limits are all measured in tokens.

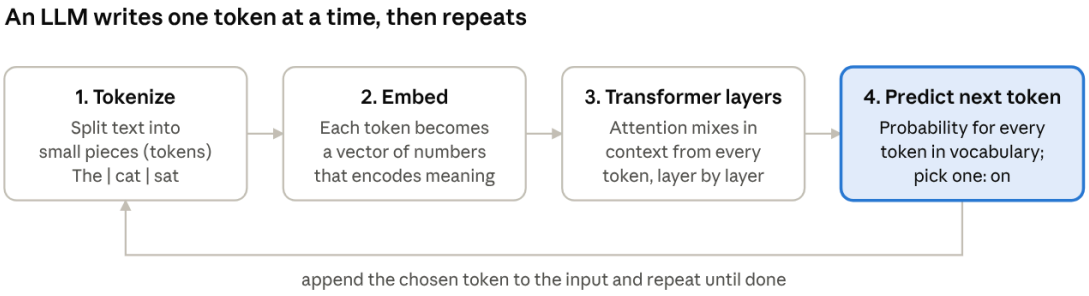
Embeddings

An **embedding** is a list of numbers (a vector) that represents a token's meaning. Words with similar meanings sit close together in this vector space, so "king" is near "queen" and far from "banana". Embeddings also power semantic search (see Part 8).

Context window

The **context window** is the maximum number of tokens a model can consider at once — your prompt, any documents, and its reply combined. Anything outside it is invisible to the model.

How generation works



Text generation loop · 4 steps, repeated per token

A language model is, at heart, a very good next-token predictor. Long answers are built one token at a time by running this loop again and again.

Training vs inference

	Training	Inference
What happens	The model learns by adjusting its parameters	The trained model is used to generate outputs
How often	Once (plus occasional updates)	Every time anyone uses it
Cost	Very high: thousands of GPUs for weeks or months	Low per request, but adds up at scale
Who does it	AI labs and research teams	Apps, developers and end users
Parameters	Change	Frozen

Key idea: text → tokens → vectors → math → probabilities → the next token. Everything else is refinement of this loop.

Part 4 — Types of generative models

Four model families do most of the generating today: autoregressive Transformers lead for text and code, diffusion models lead for images and video.

1. **GANs (Generative Adversarial Networks)** — two networks compete. A *generator* makes fakes; a *discriminator* tries to spot them. Each improves until the fakes look real.
2. **VAEs (Variational Autoencoders)** — an *encoder* compresses data into a compact "latent space"; a *decoder* rebuilds it. Sampling new points in the latent space produces new data.
3. **Diffusion models** — trained to remove noise step by step. To generate, start from pure random noise and denoise it repeatedly, guided by a text prompt, until an image appears.
4. **Autoregressive Transformers** — predict the next token from all previous tokens (Part 3). This is how GPT, Claude, Gemini and Llama generate text.

Model family	Core idea	Best at	Strengths	Weaknesses	Examples
Autoregressive Transformer	Predict next token, repeat	Text, code, chat, reasoning	Very general; scales well; follows instructions	Slow token-by-token output; can hallucinate	GPT, Claude, Gemini, Llama
Diffusion	Learn to reverse added noise	Images, video, audio	High quality and diversity; controllable with prompts	Many steps per output; compute-heavy	Stable Diffusion, DALL·E 3, Midjourney
GAN	Generator vs discriminator game	Faces, style transfer, upscaling	Fast generation; sharp images	Unstable training; less diverse outputs	StyleGAN, CycleGAN
VAE	Encode to latent space, decode back	Compression, latent spaces, anomaly detection	Stable training; smooth latent space	Blurrier outputs on their own	Used inside Stable Diffusion's pipeline

Modern systems often combine families. Stable Diffusion, for example, runs diffusion inside a VAE's latent space and uses a Transformer-based text encoder to read the prompt.

Key idea: Transformers write, diffusion models paint — and the best systems mix techniques.

Part 5 — Transformers and attention

The Transformer's breakthrough is **attention**: every token can look at every other token in the input at once and decide which ones matter for its meaning. Older models (RNNs) read text one word at a time

and forgot distant context; Transformers process the whole sequence in parallel, which is why they train fast on GPUs and handle long documents.

What attention does

Take the sentence: *"The animal didn't cross the street because **it** was too tired."* To understand "it", the model must link it to "animal", not "street". Attention computes a score between "it" and every other word; "animal" gets the highest score, so its meaning flows into the representation of "it".

Each token produces three vectors:

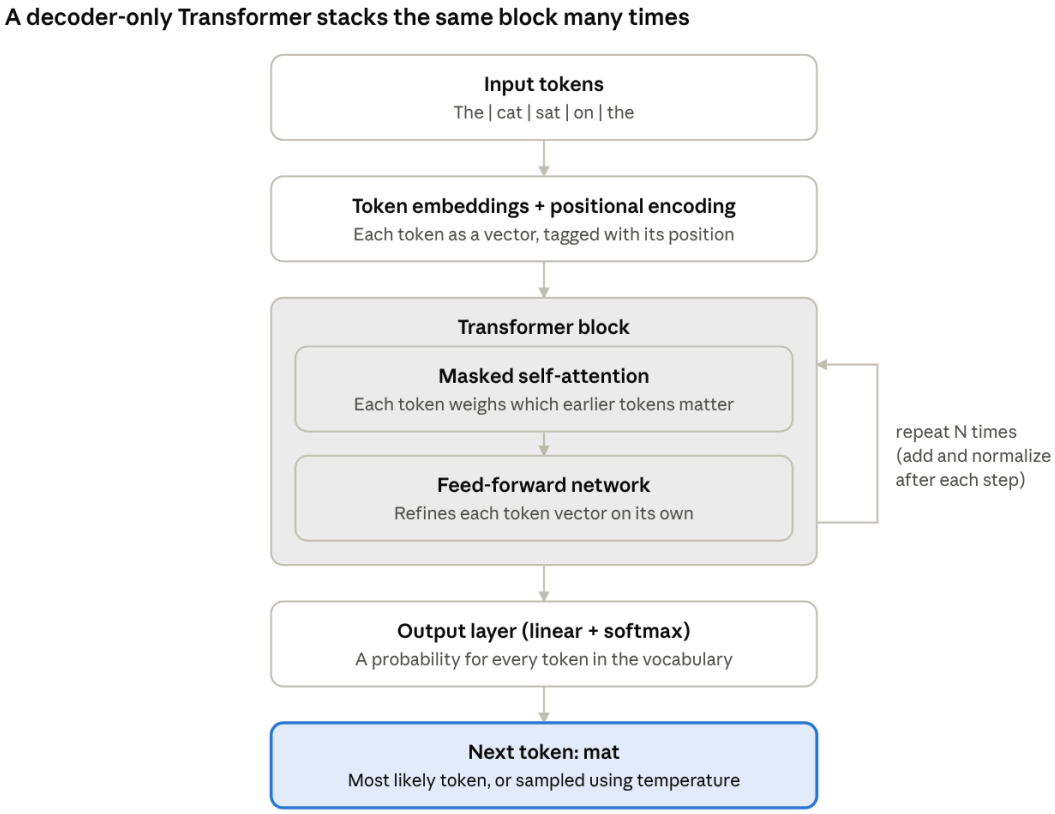
Vector	Plain-English role	Analogy (library search)
Query (Q)	What this token is looking for	Your search request
Key (K)	What each token offers	The label on each book
Value (V)	The information each token carries	The book's contents

The model compares each Query with every Key, turns the match scores into weights, and blends the Values by those weights:

```
\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{T}}{\sqrt{d_k}}\right)V
```

Multi-head attention runs many attention calculations side by side, so one head can track grammar, another track who-did-what, another track topic.

The architecture



Transformer architecture · decoder-only, as used by most LLMs

Large models repeat the Transformer block dozens of times; each layer builds a richer understanding than the one before.

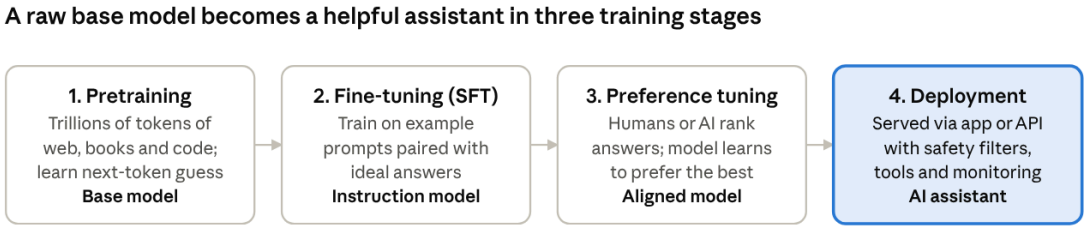
Three Transformer variants

Variant	Reads text	Typical use	Examples
Encoder-only	Whole input, both directions	Understanding: classification, search embeddings	BERT
Decoder-only	Left to right, predicts next token	Generation: chat, writing, code	GPT, Claude, Llama
Encoder-decoder	Encoder reads input, decoder writes output	Translation, summarization	T5, original Transformer

Key idea: attention lets every word consult every other word — that single mechanism is what makes modern LLMs understand context.

Part 6 — How large language models are built

A chat assistant is made in stages: pretraining gives the model broad knowledge, and fine-tuning teaches it to be helpful, honest and safe.



LLM training pipeline · 3 training stages, then deployment

Pretraining uses by far the most data and compute; the later stages use much smaller, carefully curated datasets.

The stages in more detail

Stage	Data	What the model learns	Output
Pretraining	Huge unlabeled text and code corpus	Grammar, facts, reasoning patterns, coding — by predicting the next token	Base model: completes text but doesn't follow instructions well
Supervised fine-tuning (SFT)	Thousands of prompt + ideal-response pairs written by experts	The format of helpful answers; following instructions	Instruction-tuned model
Preference tuning (RLHF, DPO, RLAIF)	Pairs of answers ranked better/worse	Which answers people prefer: helpful, harmless, honest	Aligned assistant
Reasoning training (newer)	Problems with checkable answers (math, code)	To think step by step before answering	Reasoning model

Scaling laws

Researchers found that model quality improves predictably as you increase three things together: **parameters**, **training data**, and **compute**. This is why labs keep building bigger models — though efficiency tricks (better data, distillation into smaller models, mixture-of-experts) increasingly matter too.

Open vs closed models

	Closed (proprietary)	Open-weight
Access	Via API or app only	Download and run the weights yourself
Examples	GPT, Claude, Gemini	Llama, Mistral, Qwen, DeepSeek
Pros	Usually most capable; no infrastructure to manage	Full control, privacy, customizable, can run offline
Cons	Data leaves your environment; per-use cost	You handle hosting, scaling and safety

Key idea: pretraining builds knowledge; fine-tuning builds behaviour.

Part 7 — Using GenAI well: prompting

The quickest way to get better results is a better prompt: clear context, a specific task, and an example of what good looks like.

Anatomy of a good prompt

1. **Role / context** — who the model should act as and the background it needs.
2. **Task** — exactly what to do, with an action verb.
3. **Input** — the text, data or document to work on.
4. **Constraints** — length, tone, audience, what to avoid.
5. **Output format** — bullet list, table, JSON, email.
6. **Examples** — one or two samples of the desired output.

Weak prompt	Strong prompt
"Write about climate change."	"You are a science teacher. Explain climate change to 12-year-olds in 150 words, using one everyday analogy. End with one question for class discussion."
"Summarize this."	"Summarize the report below in 5 bullet points for a busy executive. Lead with the financial impact. Report: …"

Prompting techniques

Technique	How it works	When to use	Example
Zero-shot	Ask directly, no examples	Simple, common tasks	"Translate this sentence into Hindi."
Few-shot	Give 2–5 input→output examples first	Custom formats or labels	Three sample reviews tagged positive/negative, then a new one

Technique	How it works	When to use	Example
Chain-of-thought	Ask the model to reason step by step	Math, logic, multi-step problems	"Think through it step by step, then give the final answer."
Role prompting	Assign an expert persona	Domain tone or expertise	"Act as a senior tax accountant…"
Structured output	Request a fixed format	Feeding results into software	"Return JSON with keys name, date, amount."
Prompt chaining	Split a big task into sequential prompts	Long or complex workflows	Outline → draft each section → edit
Self-critique	Ask the model to review and improve its answer	Higher-quality drafts	"Check your answer for errors, then revise."

Generation settings

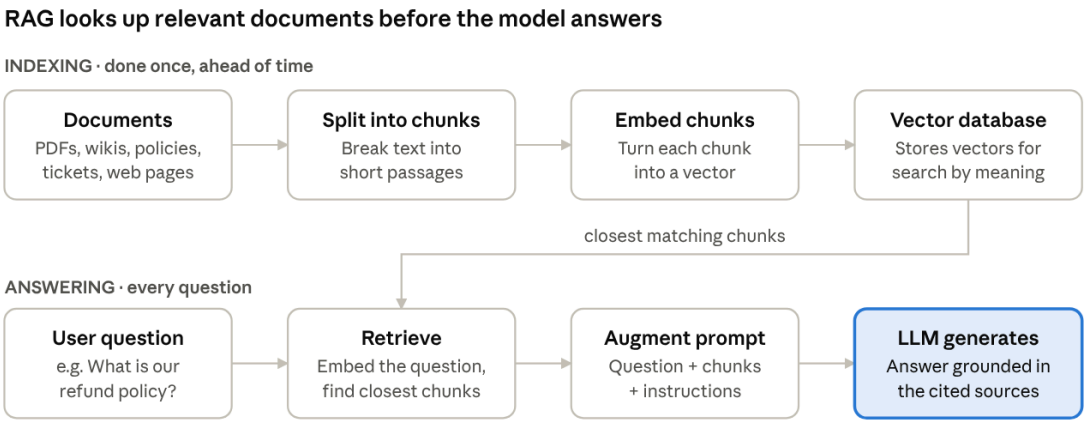
Setting	What it controls	Low value	High value
Temperature	Randomness in choosing the next token	Focused, repeatable (facts, code)	Creative, varied (brainstorms, stories)
Top-p (nucleus sampling)	Only sample from the most likely tokens whose probabilities add up to p	Safer, narrower word choice	More diverse wording
Max tokens	Upper limit on reply length	Short answers	Long answers (costs more)
System prompt	Standing instructions that shape every reply	—	—

Key idea: treat the model like a smart new colleague with no context — tell it who it is, what you need, and what good looks like.

Part 8 — Retrieval-Augmented Generation (RAG)

RAG gives a model access to *your* information at question time: it searches a knowledge base, pastes the most relevant passages into the prompt, and asks the model to answer from them. It is the most common way businesses build GenAI on private data.

Why it's needed: an LLM only knows what was in its training data, which has a cutoff date and excludes your private documents. Asked about them anyway, it may confidently invent an answer (a *hallucination*).



RAG architecture · indexing pipeline and answering pipeline

The top row runs once to build the index; the bottom row runs on every question, so answers stay current as documents change.

Tips for good RAG

- **Chunk sensibly** — split by section or paragraph so each chunk holds one idea.
- **Use hybrid search** — combine semantic (vector) search with keyword search for names, codes and IDs.
- **Re-rank** — a second model re-orders retrieved chunks by true relevance.
- **Ask for citations** — instruct the model to quote its sources and say "I don't know" when they don't contain the answer.

RAG vs fine-tuning vs prompting

Approach	What changes	Best for	Cost and effort	Keeps knowledge current?
Prompt engineering	Only the instructions	Quick wins, format and tone	Lowest	No
RAG	The context supplied at question time	Answering from private or fast-changing documents	Medium: needs an index and search	Yes — update the documents
Fine-tuning	The model's weights	Consistent style, specialised format, narrow tasks	Higher: needs training data and compute	No — retrain to update

Start with prompting, add RAG when the model needs your knowledge, and fine-tune only when you need behaviour that prompting can't achieve.

Key idea: RAG = search + generate. It turns a model with general knowledge into one that answers from your sources.

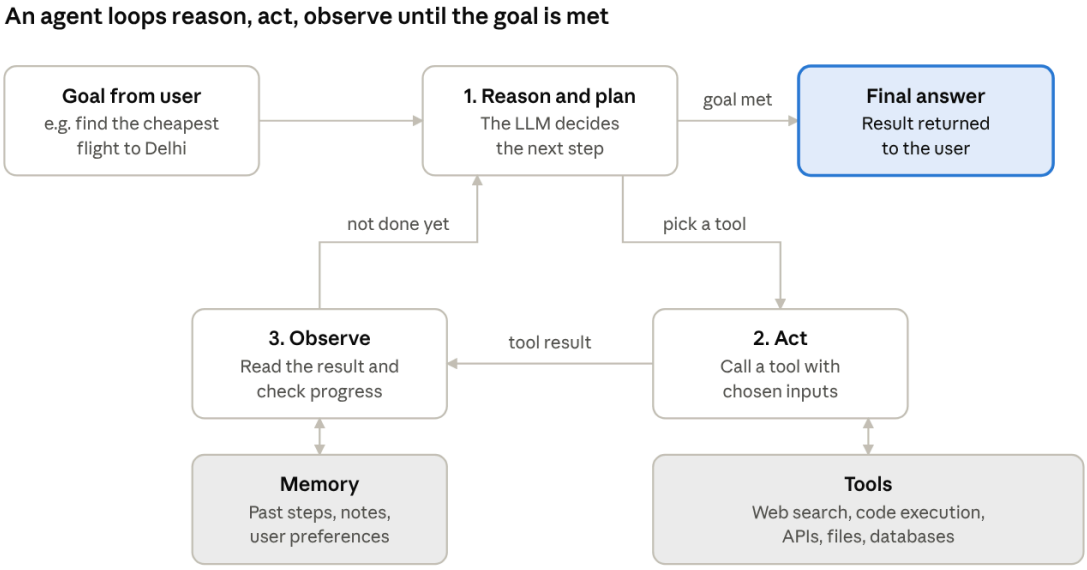
Part 9 — AI agents and tool use

An **AI agent** is an LLM that works toward a goal by taking actions, not just writing text. It decides which tool to use, runs it, looks at the result, and repeats until the job is done.

Tool use (function calling)

The developer describes available tools to the model — for example `get_weather(city)` or `search_flights(from, to, date)`. When a question needs one, the model outputs a structured request like `search_flights("HYD", "DEL", "2026-11-05")`. The application runs it and feeds the result back, and the model uses it to answer. Standards such as the **Model Context Protocol (MCP)** let one agent connect to many tools in a consistent way.

The agent loop



Agent loop · reason, act, observe, with tools and memory

Each pass through the loop is one step; complex tasks may take dozens of steps before the goal is met.

Chatbot vs assistant with tools vs agent

Level	What it does	Example	Human involvement
Chatbot	Answers from its training knowledge	"Explain photosynthesis"	Every turn

Level	What it does	Example	Human involvement
Tool-using assistant	Calls one tool when needed, then answers	Checks today's weather, then replies	Every turn
Agent	Plans and runs many steps on its own	Researches 10 sources and writes a report	Sets the goal, reviews the result
Multi-agent system	Several specialised agents coordinate	A researcher, a writer and a reviewer agent	Oversees the whole team

Common agent patterns

- **ReAct (Reason + Act)** — alternate between thinking out loud and calling tools.
- **Plan-and-execute** — write a full plan first, then carry out each step.
- **Reflection** — the agent critiques its own output and tries again.
- **Human-in-the-loop** — pause for approval before risky actions such as payments or sending emails.

Key idea: agents = LLM + tools + memory + a loop. They are powerful but need limits and human checkpoints for anything irreversible.

Part 10 — Modalities and applications

GenAI now works across text, images, audio, video and code, and the leading models are **multimodal**: one model can read an image, listen to speech and reply in text or voice.

What each modality can do

Modality	Typical input → output	Main model type	Example tools	Common uses
Text	Text → text	Autoregressive Transformer (LLM)	ChatGPT, Claude, Gemini	Writing, summarising, Q&A, translation
Code	Text or code → code	LLM trained heavily on code	GitHub Copilot, Claude Code, Cursor	Writing, explaining and debugging software
Image	Text → image; image → image	Diffusion	Midjourney, DALL·E, Stable Diffusion	Design concepts, marketing visuals, editing
Vision understanding	Image → text	Multimodal LLM	GPT-4o, Claude, Gemini	Reading charts, documents, screenshots

Modality	Typical input → output	Main model type	Example tools	Common uses
Audio and speech	Text ↔ speech; text → music	Transformer and diffusion audio models	Whisper, ElevenLabs, Suno	Voice assistants, transcription, dubbing
Video	Text or image → video	Video diffusion Transformers	Sora, Veo, Runway	Ads, explainers, storyboards

Applications by industry

Industry	Example application	Benefit
Customer service	Chat agents answering from help docs (RAG)	24/7 answers; human agents handle the hard cases
Software	Coding assistants and autonomous coding agents	Faster development and testing
Healthcare	Drafting clinical notes; summarising patient records	Less paperwork for doctors
Education	Personal tutors that explain step by step	Learning at each student's pace
Finance	Summarising reports; drafting analyses	Faster research
Marketing	Copy, images and video variants for campaigns	More content, quicker testing
Legal	Contract review and clause comparison	Faster first-pass review

Key idea: pick the model family by output type — LLMs for language and code, diffusion for images and video, multimodal models when inputs are mixed.

Part 11 — Evaluation, risks and responsible use

GenAI output is fluent but not guaranteed correct, so every serious use needs evaluation, safeguards and human review proportional to the stakes.

How models and apps are evaluated

Method	What it measures	Example
Benchmarks	Standardised skill tests	MMLU (knowledge), GSM8K (math), SWE-bench (coding)
Human evaluation	Quality as judged by people	Raters compare two answers side by side

Method	What it measures	Example
LLM-as-a-judge	A strong model grades outputs against a rubric	Scoring 1,000 support replies for accuracy and tone
Task-specific test sets	Accuracy on your own real examples	200 real customer questions with known correct answers
RAG metrics	Faithfulness to sources; retrieval relevance	Does every claim appear in the retrieved chunks?

Key risks and how to reduce them

Risk	What it looks like	Mitigation
Hallucination	Confident but false facts, fake citations	RAG with citations; ask the model to say "I don't know"; human fact-checking
Bias	Stereotyped or unfair outputs inherited from training data	Diverse test sets, bias audits, careful prompts
Privacy and data leakage	Sensitive data pasted into prompts or exposed in outputs	Enterprise plans with data controls; redact personal data
Prompt injection	Hidden instructions in a web page or file hijack the model	Treat retrieved content as data; limit tool permissions; human approval for actions
Copyright and IP	Outputs too close to protected works	Use licensed tools; review before publishing
Misinformation and deepfakes	Realistic fake text, images, audio or video	Watermarking, content provenance (C2PA), verification
Over-reliance	Accepting answers without checking	Keep a human in the loop for high-stakes decisions

Core limitations to remember

- **Knowledge cutoff** — models don't know events after training unless given search or documents.
- **No true understanding guarantee** — they predict likely text, which is usually but not always right.
- **Context limits** — very long inputs can be truncated or only partly attended to.
- **Cost and latency** — larger models are slower and more expensive per request.

Responsible-use checklist

- ☐ Is the use case appropriate for an AI that can make mistakes?
- ☐ Is a human reviewing outputs before they affect people?
- ☐ Is sensitive or personal data protected?

- ☐ Are outputs fact-checked or grounded in sources?
- ☐ Is it disclosed to users that content is AI-generated where it matters?

Key idea: trust, but verify — the higher the stakes, the more human oversight you need.

Glossary, self-check and next steps

Glossary

Term	Meaning
Agent	An LLM that takes actions with tools in a loop to reach a goal
Attention	Mechanism letting each token weigh the relevance of every other token
Context window	Maximum tokens a model can process at once (input + output)
Diffusion model	Generates images by removing noise step by step
Embedding	A vector of numbers representing meaning
Fine-tuning	Further training a model on specific data to change its behaviour
Hallucination	Confident output that is false or made up
Inference	Using a trained model to produce outputs
LLM	Large Language Model — a Transformer trained on vast text
Multimodal	Able to handle several data types, such as text and images
Parameters	The learned weights inside a neural network
Prompt	The instructions and input you give a model
RAG	Retrieval-Augmented Generation — search documents, then generate an answer from them
RLHF	Reinforcement Learning from Human Feedback — tuning a model with human preference rankings
Temperature	Setting that controls randomness of output
Token	A chunk of text (≈ ¾ of a word) that models read and write
Transformer	The neural network architecture behind modern LLMs
Vector database	A store that finds items by similarity of their embeddings

Self-check quiz

Try to answer before checking the part listed.

1. How does a generative model differ from a discriminative one? *(Part 1)*
2. Which 2017 paper introduced the Transformer? *(Part 2)*
3. Roughly how many words are in 1,000 tokens? *(Part 3 — about 750)*
4. Why do diffusion models start from random noise? *(Part 4)*
5. What are the Query, Key and Value vectors in attention? *(Part 5)*
6. What does pretraining produce, and what does fine-tuning add? *(Part 6)*
7. When would you lower the temperature? *(Part 7)*
8. Name the two pipelines in a RAG system. *(Part 8)*
9. What three steps repeat in an agent loop? *(Part 9)*
10. Give two ways to reduce hallucinations. *(Part 11)*

Next steps

Level	What to do next
Practise	Use an AI assistant daily; rewrite the same prompt three ways and compare results
Build	Call an LLM API from a short Python script; then build a small RAG app over your own PDFs
Go deeper	Read "Attention Is All You Need"; take a free course such as DeepLearning.AI's short courses or Hugging Face's LLM course
Advanced	Fine-tune a small open model with LoRA; build an agent with tool calling and evaluate it on a test set